

**Measuring flow control
under pressure.**

Request path

Each tenant tags its requests with a tier and a tenant id, flow control orders them by priority, and one shared vLLM pool serves the result.

Request path

ENANTS

premium
priority 100

standard
priority 0

batch
priority -10

ENDPOINT PICKER

flow control
priority-aware admission queue

100

0

0

-10

admits by priority while the pool has room;
queues instead of rejecting when it does not

unmatched header or unsaturated pool: every tier logs priority 0, and nothing queues

SHARED POOL

vLLM, 1x H100, TP 1
openai/gpt-oss-20b

max-num-seqs 128

max-num-batched-tokens 8192

max-model-len 32768

gpu-memory-utilization 0.90

prefix caching off

er header x-llm-d-inference-objective
nant header x-llm-d-inference-fairness-id

How priority is decided

Four stages turn a header into a priority, test whether the pool is saturated, and, only then, order the queue.

How priority is decided

1 / 2

1 · REQUEST ARRIVES

x-llm-d-inference-
objective

its tier

x-llm-d-inference-
fairness-id

2 · RESOLVE PRIORITY

Endpoint Picker binds
the objective to an
InferenceObjective:

premium 100

standard 0

batch -10 · unbound → 0

3 · SATURATED?

utilization detector =
max(QueueDepth / 4,
KVCache / 0.8)

below the 1.0 ceiling → pass

at the ceiling → queue

4 · ORDER

across bands:
strict priority

100 → 0 → -10

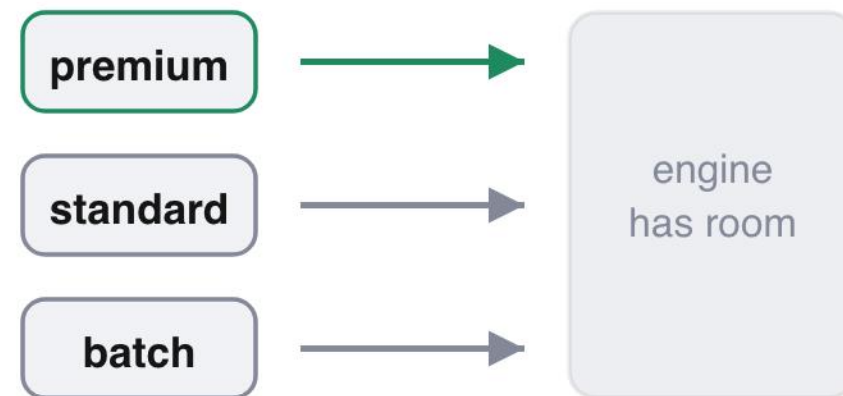
within a band:
round-robin

How priority is decided

2 / 2

A · POOL HAS CAPACITY

detector below 1 · queue empty



Priority is resolved but never used.

All three tiers dispatch on arrival. Stage 4 is skipped; flow control is invisible.

B · SATURATED, MIXED TIERS

detector ≥ 1 · strict priority

queue, front $\rightarrow$ back



Premium jumps the queue.

Every opening in the running batch goes to the highest band first: premium, then standard, then batch waits longest.

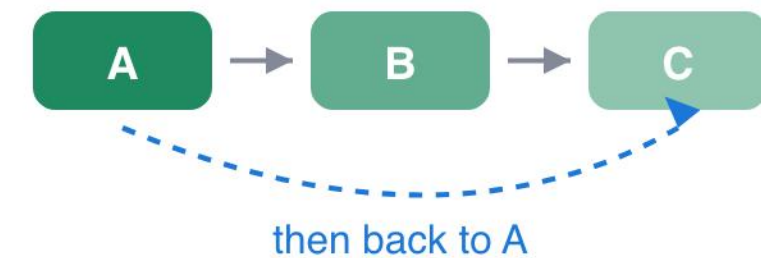
This is why premium stays ahead:

p95 1117 ms vs 1406 ms at 128 out.

C · SATURATED, SAME BAND

three tenants, all priority 100

round-robin, taking turns



No band jumps another.

All three sit at priority 100, so ordering rotates whose request dispatches next, A then B then C.

Bounds a bursty tenant's share.

A cannot crowd out B and C by flooding; the turn always comes.

The knobs

Six settings decide how much load reaches the engine and when flow control calls the pool saturated.

The knobs

	OWNER	WHAT IT DOES	SET TO
<code>x-num-seqs</code>	vLLM engine	Max requests the engine runs at once, the running-batch ceiling	128
<code>queueDepthThreshold</code>	utilization detector	Queue length that counts as saturated	4
<code>CacheUtilThreshold</code>	utilization detector	KV-cache fraction that counts as saturated	0.8
<code>xConcurrency</code>	concurrency detector	Hard cap on in-flight requests; the only knob that reduces load on the engine	swept 32–12
<code>maxTotalNumSeqsInFlight</code>	concurrency detector	Allowed burst above the cap	0 (default)
<code>priority bands</code>	flow-control ordering	The dispatch order, not a detector: premium 100, standard 0, batch –10	100 / 0 / –

How we chose the operating point and configuration

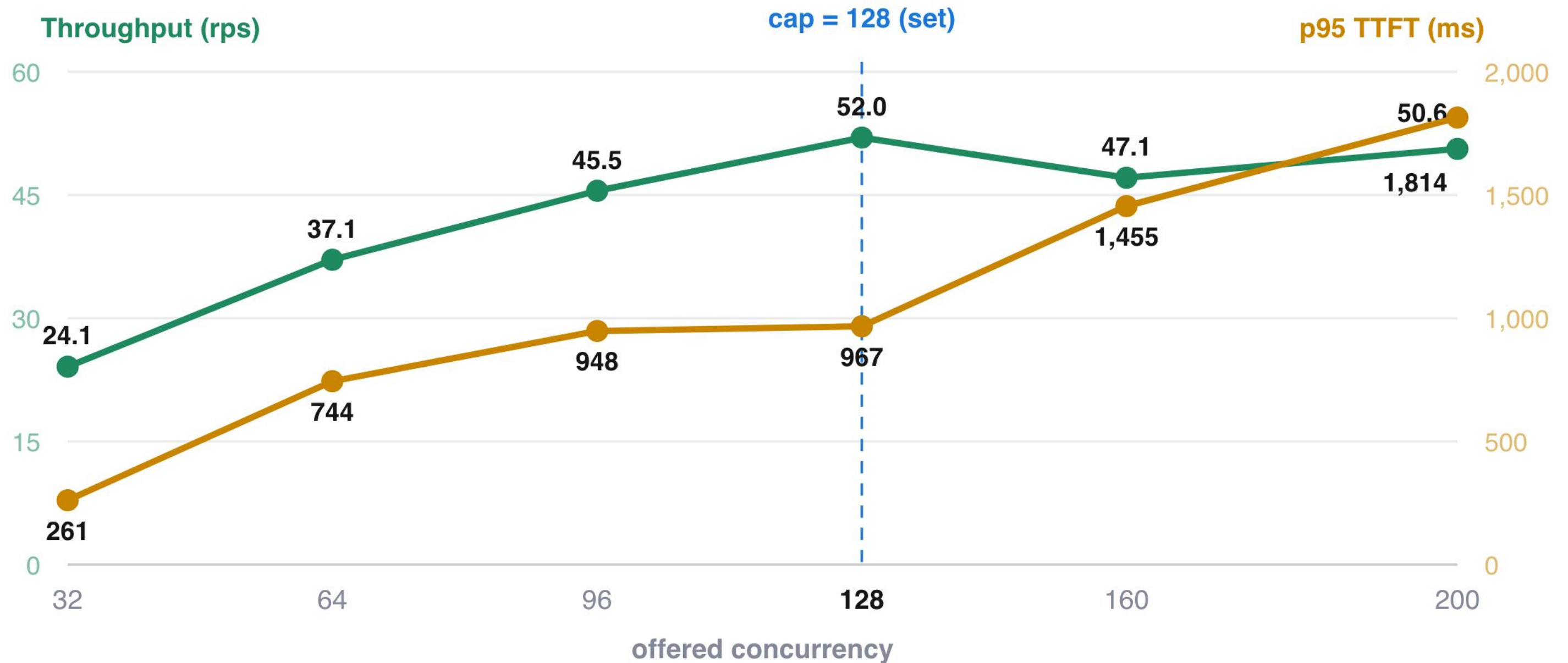
Flow control queues populate only at saturation, so before the experiments we sweep the model server with a single tenant to find the saturation point.

How we chose the operating point and configuration

1 / 2

Operating point: throughput peaks at 128, latency climbs past it

Single tenant, 512 in / 128 out, 180 s per point. Two passes, reversed order, agreed.



How we chose the operating point and configuration

Three request shapes, one swept dimension

Input 512 tokens, held constant. Output is the swept dimension.

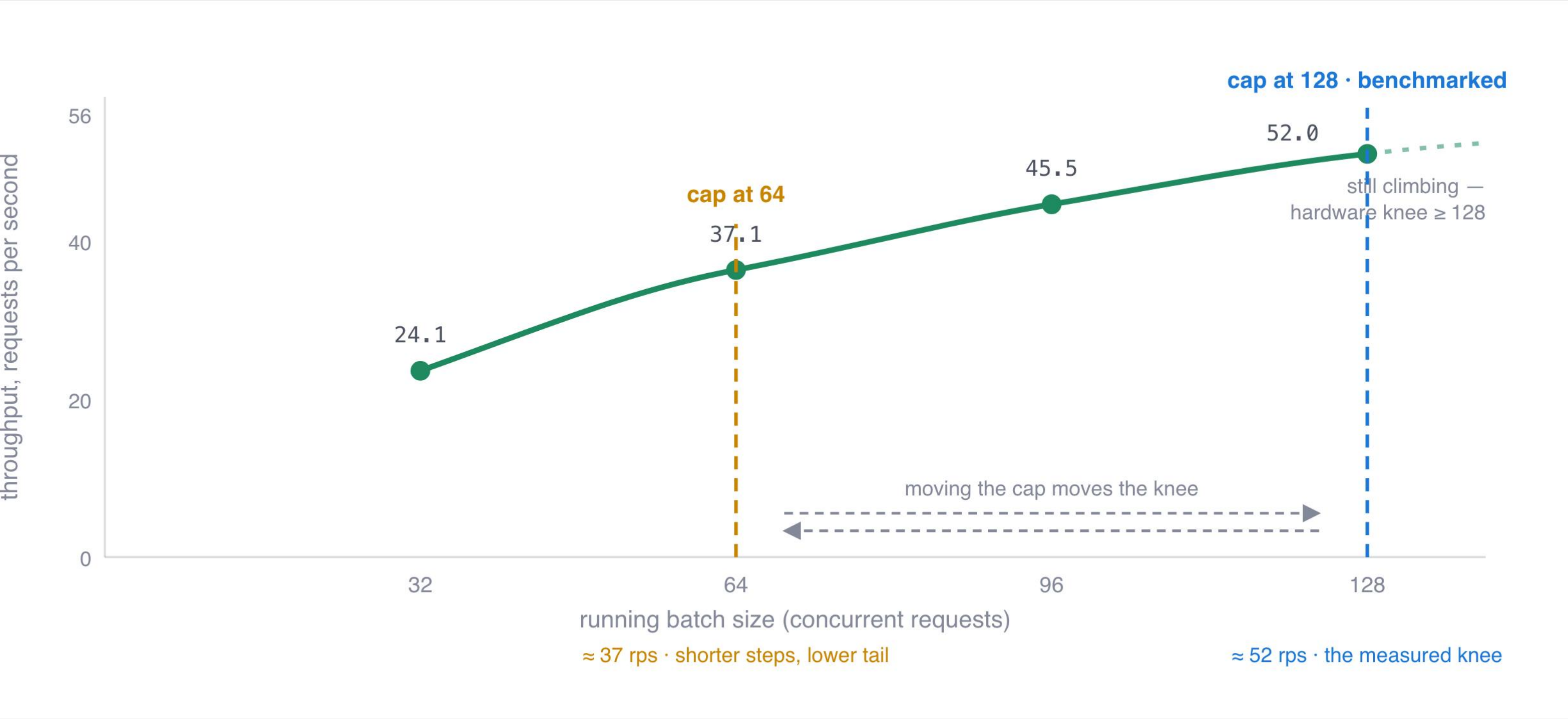


CHOICE	VALUE	WHY
Input length	512 in (fixed)	Short interactive work; held constant across every scenario
Output length	64 / 128 / 512 (swept)	Output is the saturation limiter, so we swept it; 128 is the headline
max-num-seqs	128 (set)	The engine's running-batch cap we set; the sweep confirmed the knee lands here
max-num-batched-tokens	8192	Prefill budget per step for this model and GPU
Prefix caching	off	Latencies reflect scheduling, not a warm cache
Repeats	3 counted	Each headline is the median of the three per-repeat p95s, steady-state trimmed, with the min-max range shown

The knee is a choice

Would the knee sit at 64 if we set max-num-seqs to 64?

The knee is a choice



Setup and verification

All runs follow the same basic setup.

Setup and verification

	VALUE
el	openai/gpt-oss-20b
lware	one H100, TP 1
m-seqs	128
m-batched-tokens	8192
del-len	32768
mory-utilization	0.90
_prefix_caching	False
t / output tokens	512 in / 64,128,512 out

	CARRIES
d-inference-objective	the priority tier, premium, standard, or batch
d-inference-fairness-id	the tenant, used for same-band fairness

	RESOLVES TO
mium	100
dard	0
h	-10

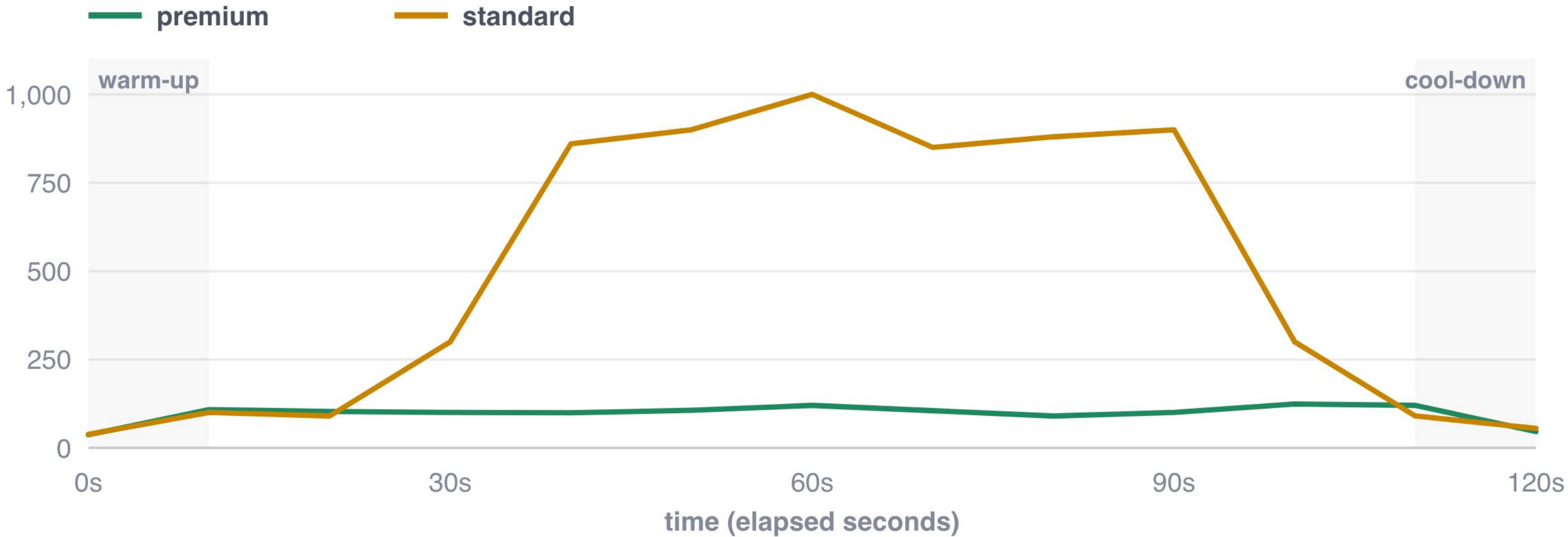
	WHERE	PASSES WHEN
ity routing	Endpoint Picker /metrics	<code>queue_duration_seconds{priority="100"}</code> present for premium
saturated	vLLM /metrics	<code>num_requests_running</code> pinned at 128 and <code>num_requests_waiting</code> > 0

Service tiers

When the pool saturates, priority decides who waits.

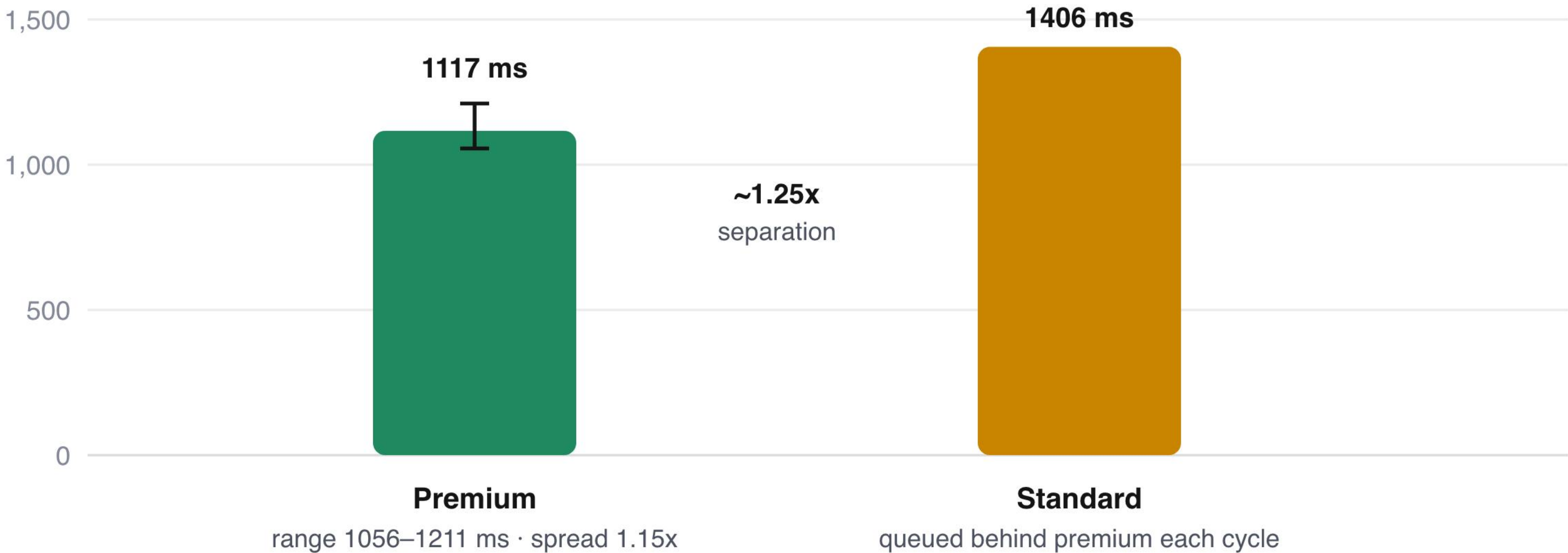
Offered load over the run: standard surges, premium holds

In-flight requests per tier. Standard floods mid-run while premium keeps a steady low rate.



Service tiers, flow control on: premium clears first on a saturated pool

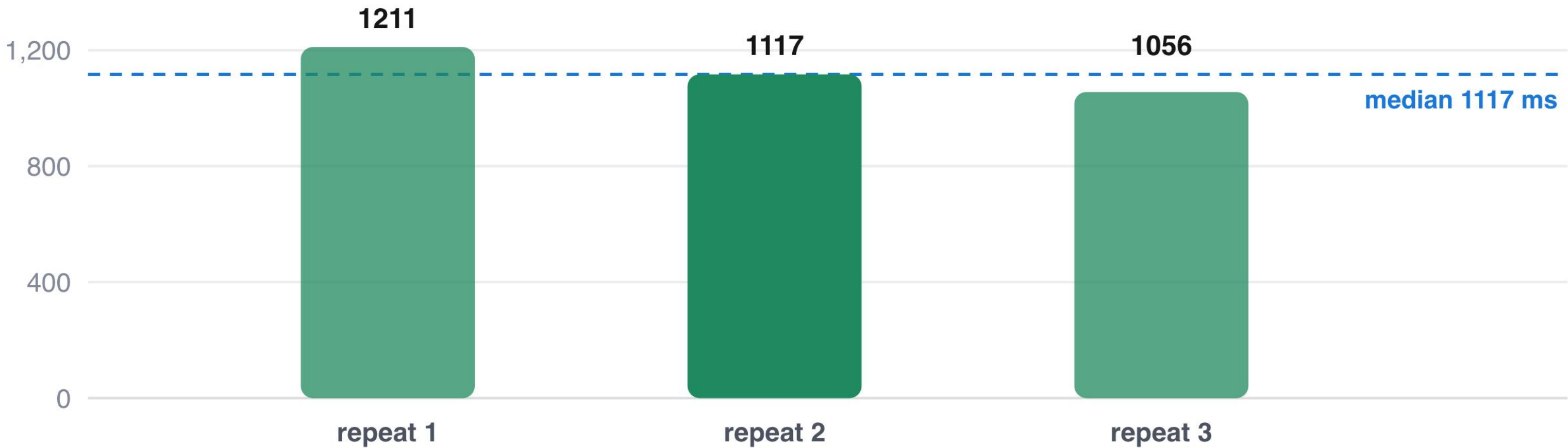
p95 TTFT, median of three per-repeat percentiles, 512 in / 128 out, steady-state trimmed.



Premium p50 is 374 ms while the pool stays pinned at its 128-slot cap with a real queue behind it.

Premium p95 TTFT per repeat

Each repeat's own steady-state p95; the headline is the median of the three.



The spread across repeats is 1.15x, inside the $\leq 1.5x$ stability gate this round applies to every headline.

Tier protection holds on two replicas

The same service-tiers scenario on a two-replica pool with flow control on kept premium ahead of standard, so the ordering survives scale-out.

Batch isolation

A batch flood past the pool's capacity is queued instead of rejected, and interactive latency is unharmed.

Offered load over the run: batch ramps in behind the interactive tiers

In-flight requests per tier. Batch floods the pool after the interactive tiers are established.



Rejected requests under a batch flood

A low-priority batch tenant floods a saturated pool. Flow control off sheds the overflow with HTTP 429s. Flow control on queues it instead.

48,224



FLOW CONTROL OFF

requests rejected with 429

queued



0



FLOW CONTROL ON

requests rejected

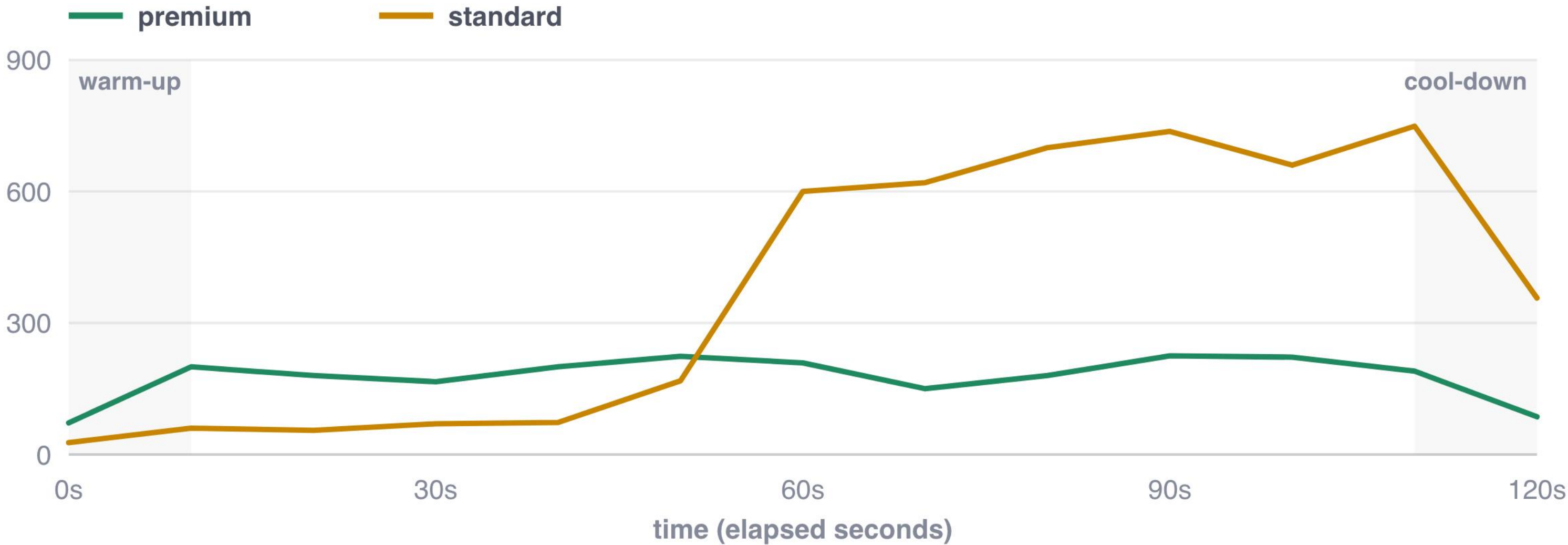
Consolidation

Two premium tenants share one GPU at higher utilization while a lower-priority tenant floods the pool.

Consolidation

Offered load over the run: two premium tenants steady, standard floods

In-flight requests. Two premium tenants hold a packed steady load; a standard tenant floods mid-run.



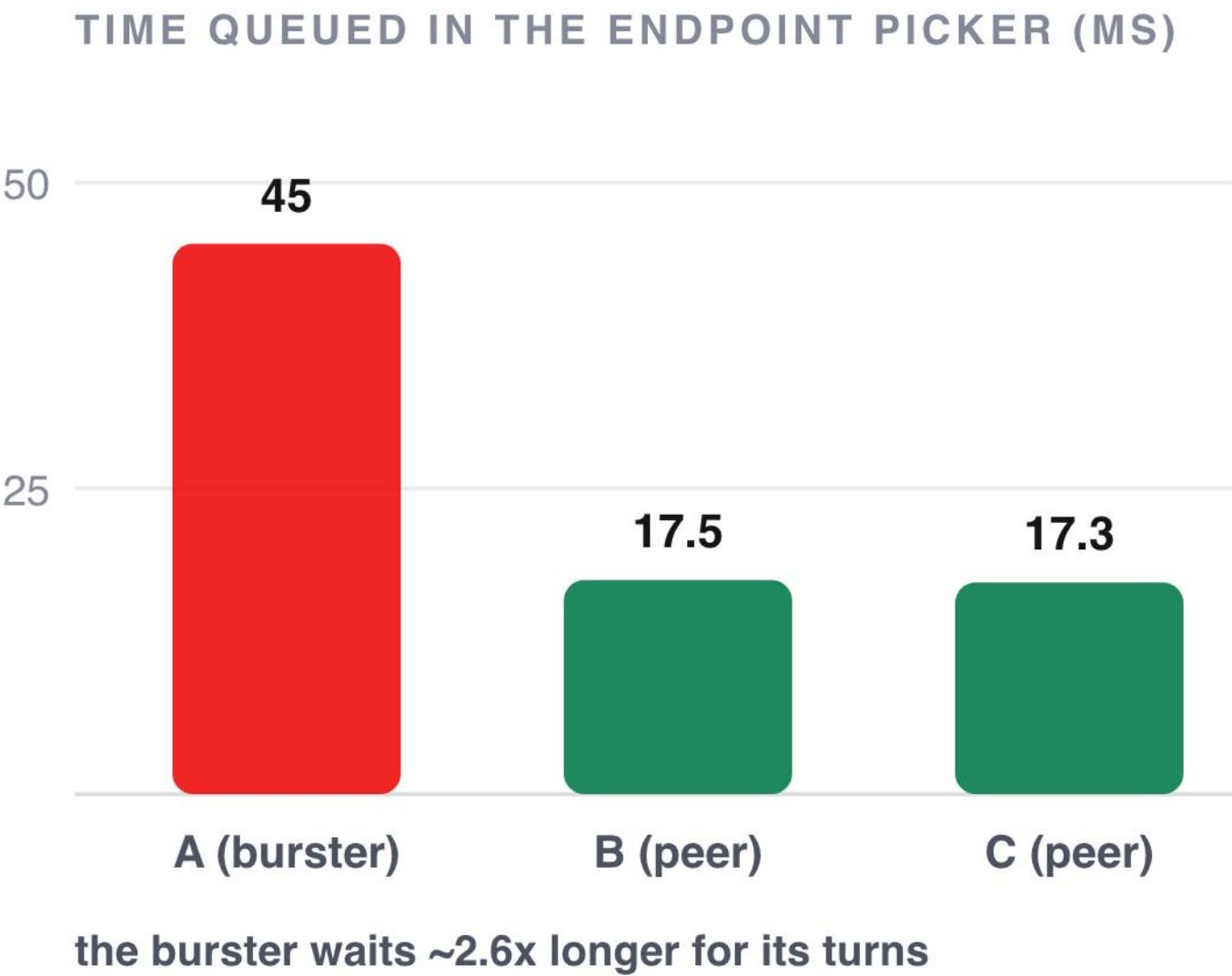
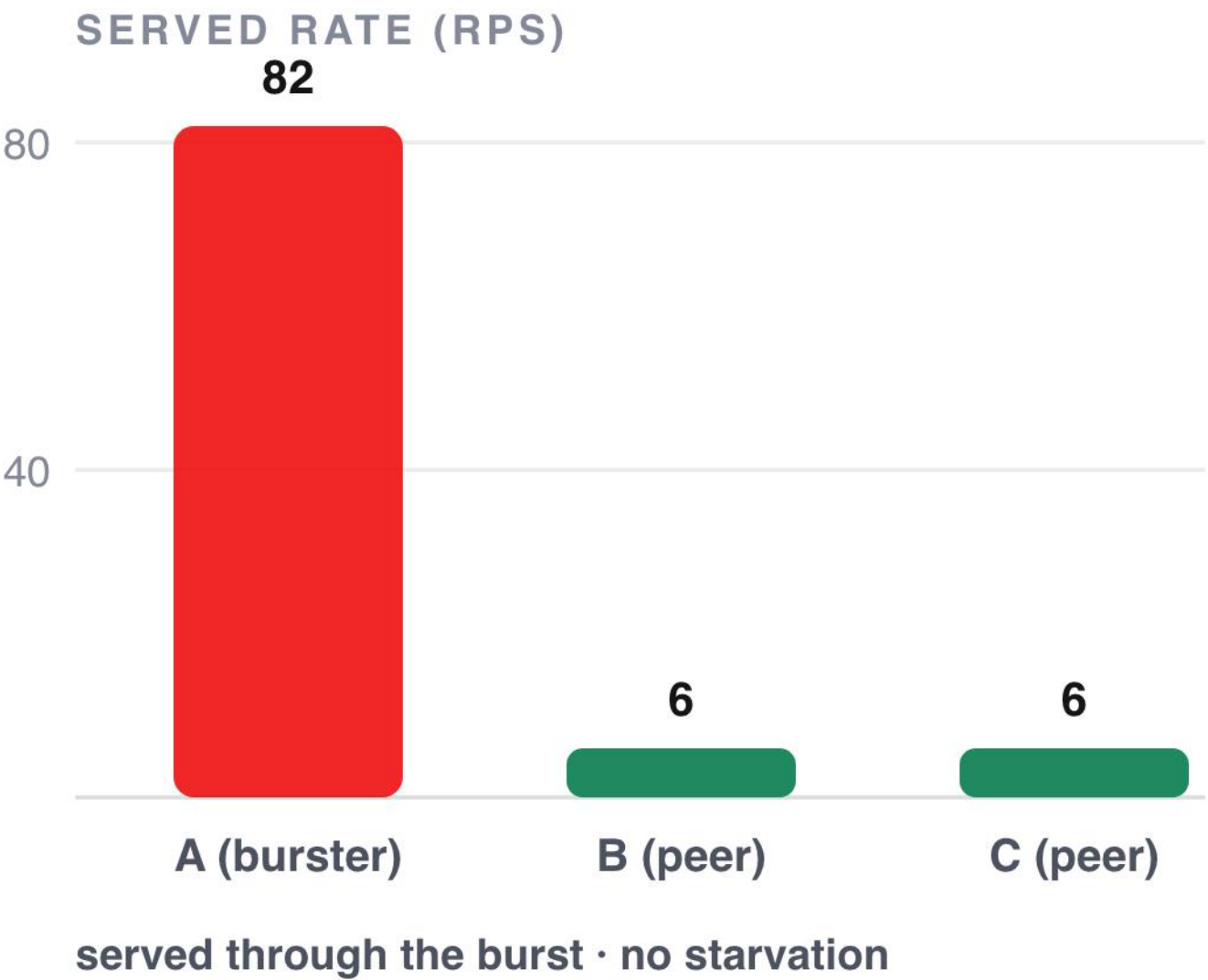
Same-band fairness

Within one priority band, round-robin bounds a bursty tenant's share of dispatch turns.

Same-band fairness

Fairness under a big burst: the burster is served, its excess waits

Three tenants, one band, flow control on. Median of three 300 s repeats, steady-state trimmed.



Two saturation detectors

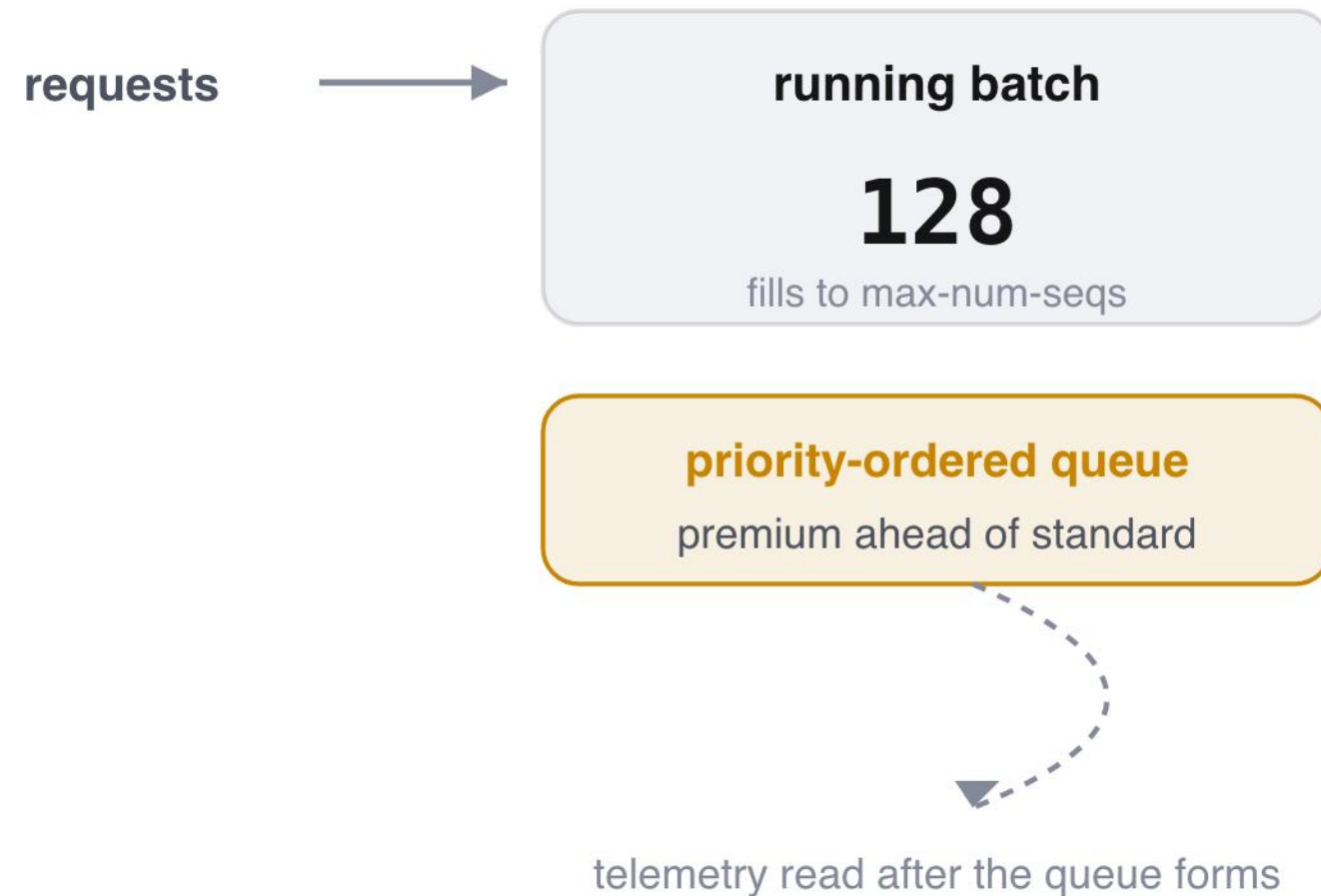
Both detectors gate the same flow-control admission queue.

Two saturation detectors

1 / 2

UTILIZATION DETECTOR (SHIPPED)

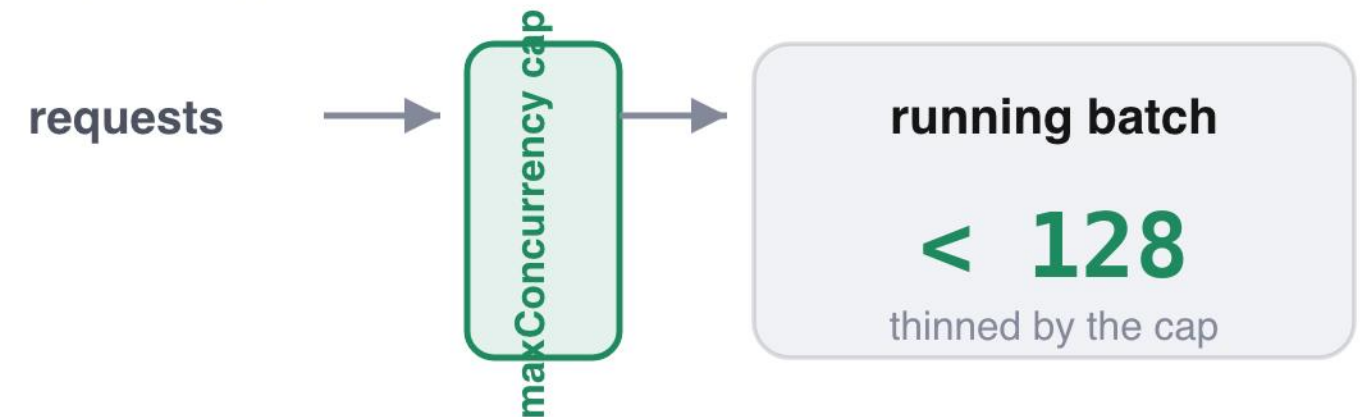
reads queue depth + KV telemetry



Decides who waits.
Cannot cap the batch.

CONCURRENCY DETECTOR (UPSTREAM)

caps in-flight load before the engine



synchronous: reacts before telemetry updates

blind to KV pressure, so long generations can fill the cache and force preemption unseen

Caps in-flight load.
Thins the batch.

Two saturation detectors

Concurrency detector: turning maxConcurrency moves the pain

Same load at every cap. Premium bottoms at maxConcurrency 48; standard improves as the cap loosens.

